LibraryDeskSense

An IoT-based Monitoring System for Library study Desks

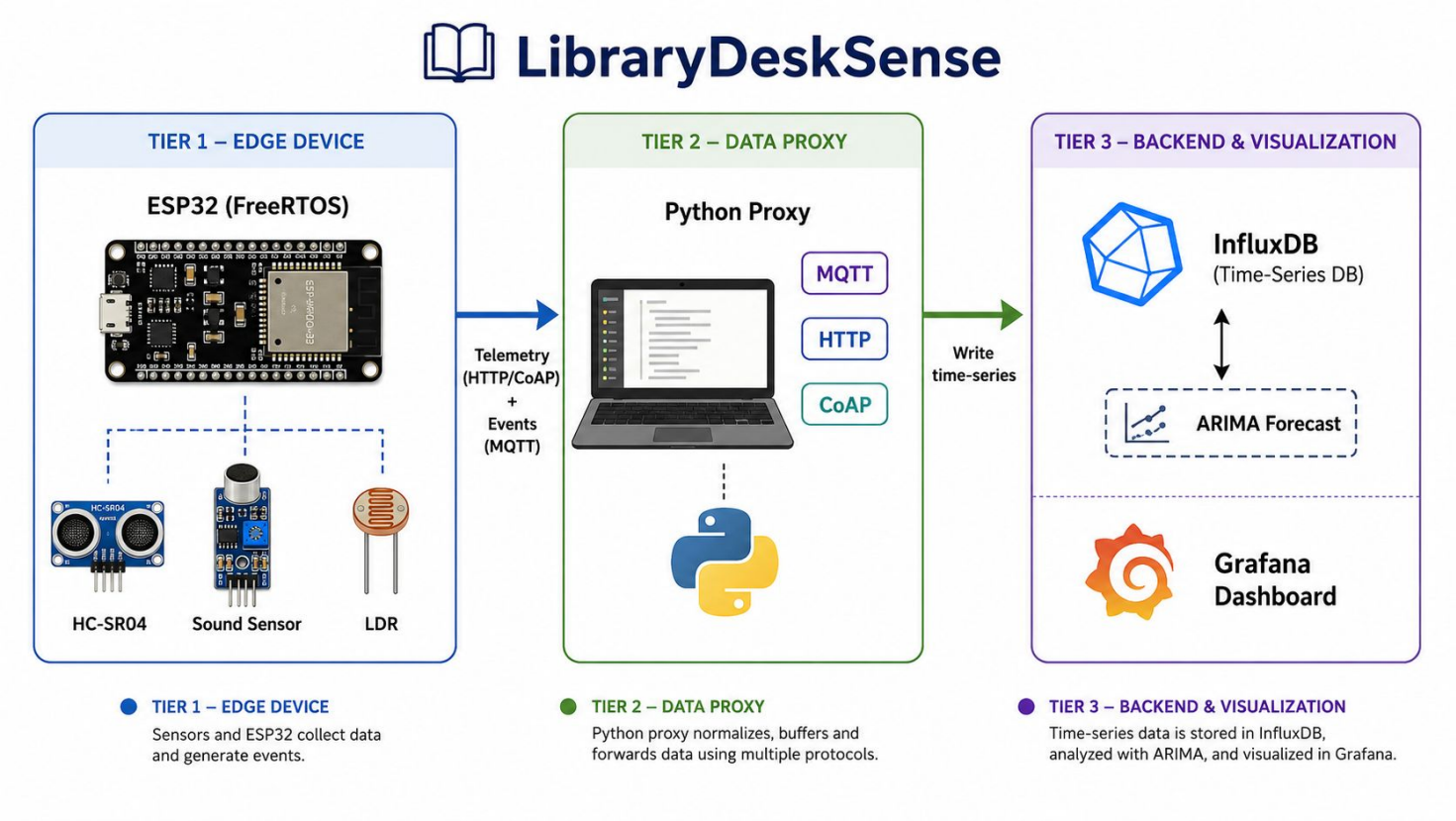
Hedi Hechmi
Electronics for Intelligent Systems, Big Data and Internet of Things

July 7, 2026

Problem and Motivation

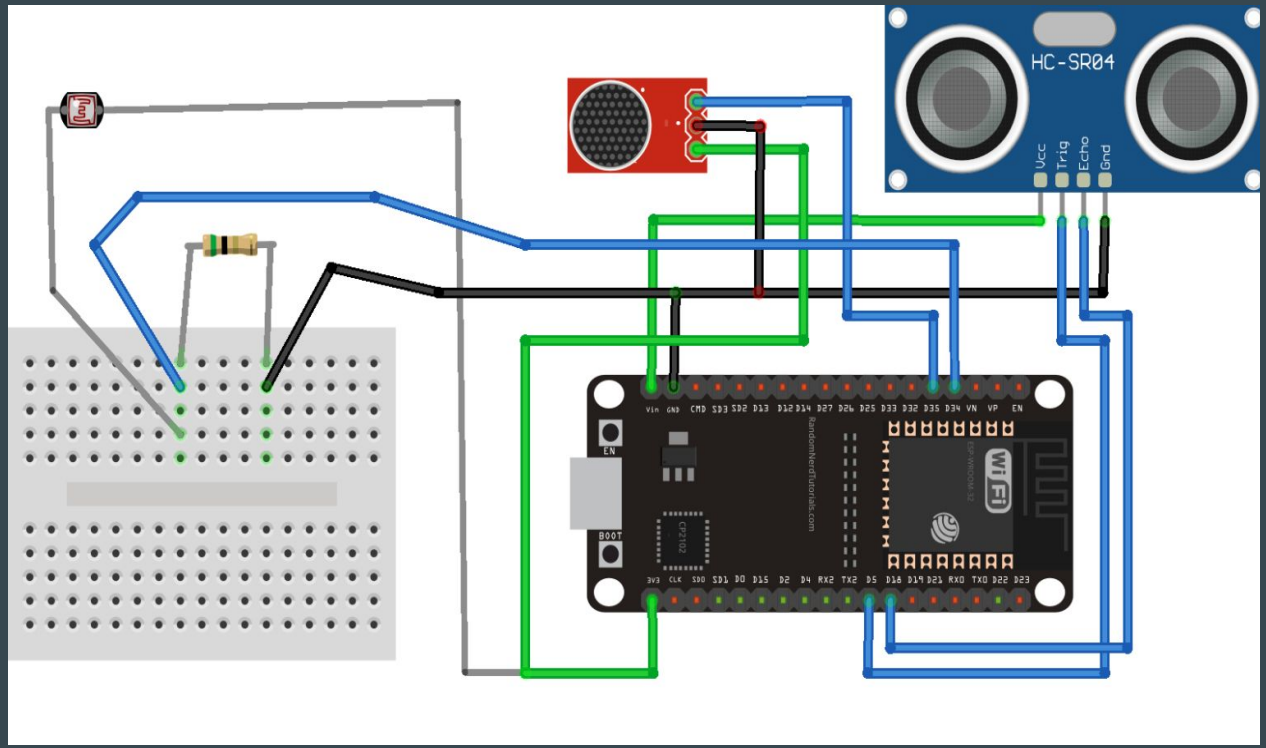
- Libraries and study rooms are shared spaces where finding the right desk is a challenge
- Students want to know:
  - Is a desk free or occupied?
  - Is the area quiet enough to study?
  - Is the lighting good for reading?
- Traditional solutions only count people at the entrance — no desk-level info
- Our goal: build a low-cost IoT system that monitors each desk in real time

System Architecture



Hardware setup

- ESP32 DevKit V1 microcontroller (Wi-Fi + Bluetooth, 240 MHz, 4 MB flash)
- HC-SR04 ultrasonic sensor → occupancy detection (GPIO 5 / 18)
- Analog sound sensor → noise level via ADC (GPIO 35)
- LDR + voltage divider → light intensity via ADC (GPIO 34)
- 12-bit ADC (0–4095) with 12 dB attenuation for full 3.3 V range



Circuit Diagram

ESP32 Firmware — FreeRTOS Architecture

- Built with ESP-IDF v5.3.5 (no Arduino) — 4 concurrent FreeRTOS tasks
- Occupancy Task (1s) — reads HC-SR04 distance, sets occupied flag
- Noise Task (500ms) — 100 rapid ADC samples, computes peak-to-peak
- Light Task (2s) — single ADC read from LDR voltage divider
- Event Task (configurable) — publishes telemetry + detects events
- 2 Mutexes protect shared resources:
  - data_mutex → shared sensor variables
  - adc_mutex → ADC1 hardware access
- Snapshot pattern: brief lock → copy to local vars → unlock → work

Communication Protocols — MQTT, HTTP, CoAP

- Three protocols, each with a specific role:
- MQTT (TCP) — asynchronous events + runtime configuration
  - Publish: telemetry, occupancy, noise, lighting events
  - Subscribe: 5 config topics (thresholds, sampling rate, mode)
- HTTP (TCP) — telemetry via POST /telemetry
  - Reliable, heavy headers (~200–500 bytes)
- CoAP (UDP) — telemetry via non-confirmable POST
  - Lightweight (~15–50 bytes), fast, connectionless
- Runtime protocol switching via MQTT topic:

  desk/config/communication_mode → "mqtt" | "http" | "coap"

Runtime Configuration via MQTT

- All key parameters can be changed live — no reflash needed
- 5 subscribed configuration topics:
  - desk/config/sampling_rate → event loop period (ms)
  - desk/config/noise_thr → high-noise threshold
  - desk/config/light_thr → poor-lighting threshold
  - desk/config/occupancy_thr → distance threshold (cm)
  - desk/config/communication_mode → mqtt / http / coap
- Example (change noise threshold to 300):

  mosquitto_pub -h localhost -t desk/config/noise_thr -m "300"
- ESP32 MQTT callback parses payload and updates global variable
- Enables live demo, adaptive tuning, and protocol switching

Backend — Python Proxy, InfluxDB, Grafana

- 3-tier backend, all running on my laptop
- Python Proxy (proxy.py):
  - MQTT subscriber → listens to desk/# topics
  - Flask HTTP server → POST /telemetry endpoint (port 8080)
  - Writes every message to InfluxDB
- InfluxDB 2.7 — time-series database
  - Org: LibraryDeskSense | Bucket: library_desk
  - Measurements: telemetry, event, forecast
- Grafana 11.4 — visualization dashboard
  - Flux queries against InfluxDB
  - 6 panels, auto-refresh every 1s
- Mosquitto 2.1 MQTT broker — runs as Windows service

Grafana Dashboard — Real-Time Monitoring

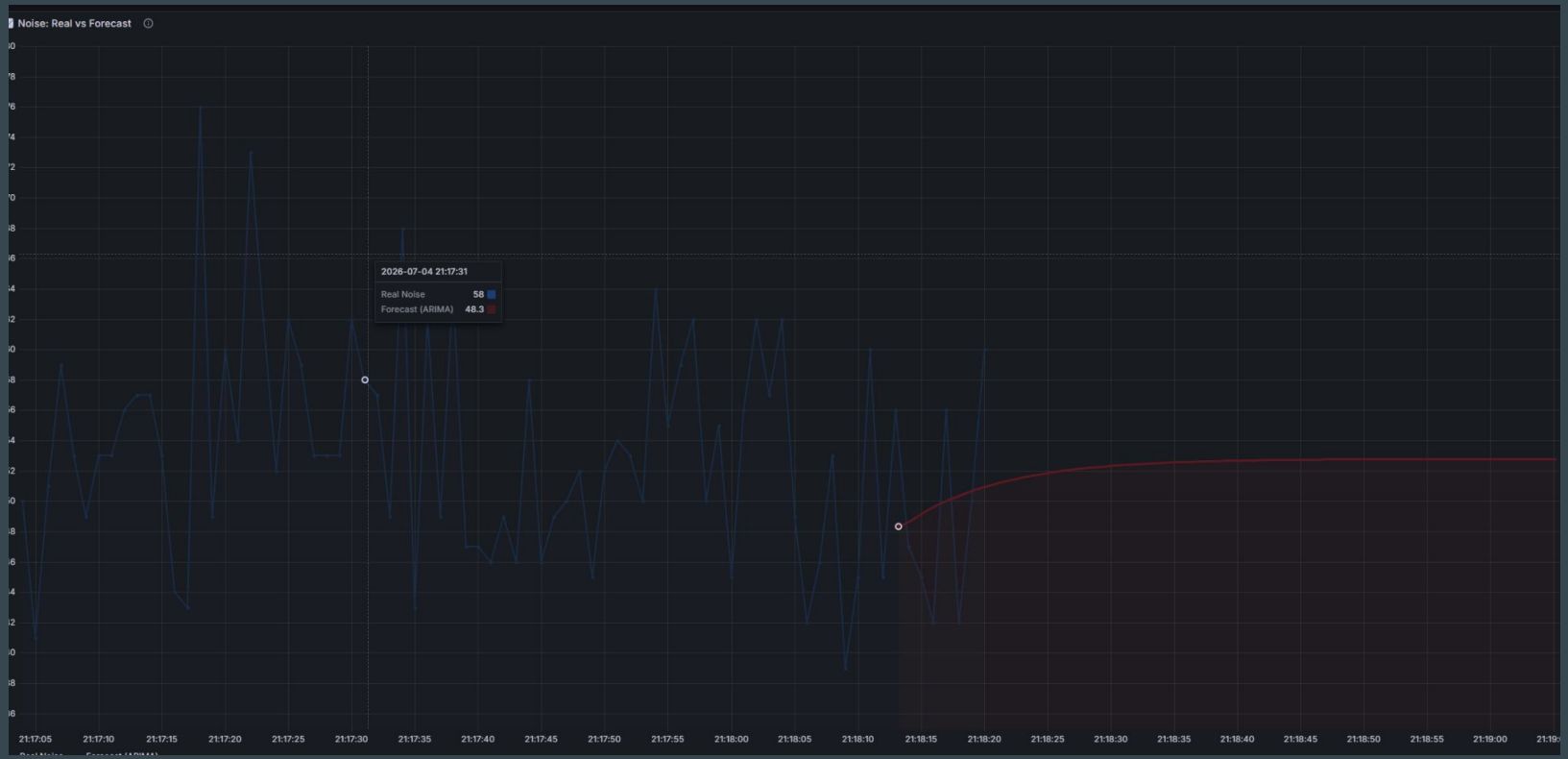
- 6 live panels, refreshing every 1 second
- Desk Status — OCCUPIED 🪑 (red) / FREE ✅ (green)
- Recent Events — colored table (occupied, released, high noise, poor light)
- Noise Level — time series with threshold line at 200
- Light Intensity — time series with threshold line at 1000
- Desk Distance — ultrasonic reading in cm
- Noise: Real vs Forecast — actual data + ARIMA prediction overlay



Grafana dashboard

Forecasting — ARIMA Model for Noise Prediction

- Python module (forecast.py) runs every 60 seconds
- Reads last 30 minutes of noise data from InfluxDB
- ARIMA(2,1,2) — 2 AR terms, 1 differencing, 2 MA terms
- Train/Test split: 80% / 20% for evaluation
- Metrics on test set:
  - MAE ≈ 16.5 (avg error in noise units)
  - MSE ≈ 409.6
- Forecast: 300 future points → written back to InfluxDB
- Old forecasts deleted before each cycle (avoids overlap)
- Displayed live in Grafana: Real vs Forecast overlay



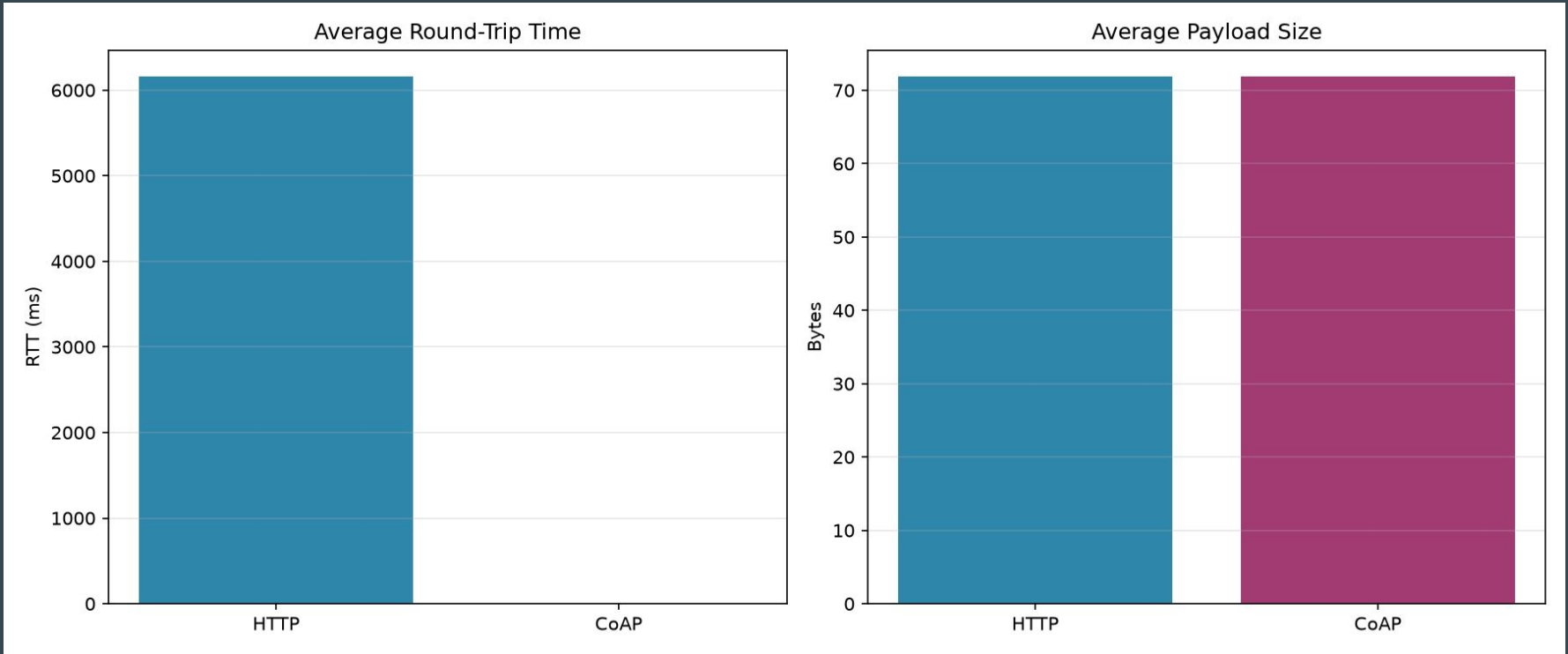
Real vs Forecasting

Communication Overhead — HTTP vs CoAP

- Python benchmark (benchmark.py) — 100 messages per protocol
- Identical JSON payload for fair comparison
- Metrics measured: RTT, payload size, success rate
- Results (100 messages, localhost):

Metric	HTTP	CoAP
Success rate	100%	100%
Payload (bytes)	71.9	71.9
Avg RTT	6161 ms	2.27 ms
Median RTT	6155 ms	1.78 ms

- CoAP is ~2700× faster than HTTP
- Both deliver 100% of messages, same payload size
- Trade-off: HTTP = reliable + rich headers, CoAP = fast + lightweight



Benchmark comparison

Conclusion

- Delivered a fully working IoT system covering all requirements:
  - ✔ ESP32 firmware with FreeRTOS (4 tasks, 2 mutexes)
  - ✔ 3 sensors: occupancy, noise, light
  - ✔ 3 protocols: MQTT, HTTP, CoAP
  - ✔ Runtime configuration via MQTT (5 parameters)
  - ✔ Python proxy + InfluxDB + Grafana (6 panels)
  - ✔ ARIMA forecasting with MAE/MSE evaluation
  - ✔ HTTP vs CoAP benchmark
- Key findings:
  - CoAP ~2700× lower RTT than HTTP (same payload, same reliability)
  - ARIMA effective as short-term baseline for noise prediction
- LibraryDeskSense demonstrates a complete end-to-end IoT pipeline:
  from embedded sensing → to data storage → to visualization and analysis